

Coding & Solution

OptiCrop — Smart Agricultural Production Optimization Engine

Development Process

- Built and validated entirely inside a single Jupyter notebook first (EDA through model saving), confirming each stage's output before moving to the next.
- Flask backend was developed only after the model artifacts were confirmed working via a standalone `predict_crop()` test function in the notebook.
- Each new HTML page and Flask route was tested manually in the browser immediately after being written, rather than batching untested changes.

Core Solution Logic (app.py — prediction route)

On POST request, the form values (N, P, K, temperature, humidity, pH, rainfall) are collected, converted to floats, assembled into a NumPy array, transformed using the saved `StandardScaler`, passed to the saved Random Forest model's `predict()` method, and the resulting numeric class is decoded back into a crop name using the saved `LabelEncoder` before being rendered back to the user.

Error Handling

Form submission is wrapped in a `try/except` block; invalid or missing input values return a clear error message on the same page rather than crashing the server.